# Project Documentation – Student Performance Analytics System

**EWB Courses | Python with AI – Project Assignment**

## 1. Student Performance Analytics System

| Field | Details |
|---|---|
| **Made By** | Pranav Thakur |
| **Technology Used** | Python, NumPy, Pandas |

## 2. Objective

Teachers and institutes keep student records (marks, attendance) in tables, but reading raw numbers does not directly answer questions such as *who is the topper?*, *how many students failed?* or *which subject is the weakest?*

This project was created to solve that problem. It reads student data from a CSV file, processes it with Python, Pandas and NumPy, and presents clear results:

- Total and average marks of every student
- Grade and Pass/Fail status
- Highest, lowest and overall class performance
- Subject-wise performance
- Top-performing students
- Attendance analysis, including students with low attendance

It also helped me practise functions, conditions, loops, NumPy and Pandas in one complete, working project.

## 3. Technologies Used

| Technology | Purpose in this project |
|---|---|
| **Python 3** | Main programming language (functions, conditions, loops) |
| **Pandas** | Loading the CSV, DataFrame operations, filtering, sorting, column calculations |
| **NumPy** | Numerical and statistical calculations (mean, max, min, standard deviation) |
| **CSV file** | Storing the student records |
| **Git & GitHub** | Version control and project submission |
| **VS Code** | Code editor |

Only Pandas and NumPy are needed as external libraries (listed in `requirements.txt`). This project focuses on Python fundamentals, Pandas, and NumPy. Machine learning is not included in the current version.

## 4. Dataset Description

The dataset is stored in `data/students.csv` and contains **20 student records** (all from the BCA department).

| Column | Description |
|---|---|
| `Student_ID` | Unique ID of the student (ST001 – ST020) |
| `Name` | Name of the student |
| `Department` | Department of the student (BCA) |
| `Python Marks` | Marks obtained in Python (out of 100) |
| `Data Structures Marks` | Marks obtained in Data Structures (out of 100) |
| `Java Marks` | Marks obtained in Java (out of 100) |
| `Attendance` | Attendance percentage |

**Sample rows:**

```
Student_ID,Name,Department,Python Marks,Data Structures Marks,Java Marks,Attendance
ST001,Rahul Sharma,BCA,85,78,82,92
ST002,Priya Singh,BCA,91,88,90,95
ST003,Aman Kumar,BCA,67,72,70,84
```

The dataset contains a realistic mix of high, average and low performers so that every type of result (A+ grades, Fail cases, low attendance) can be demonstrated.

## 5. Implementation

### Project structure

```
Student-Performance-Analytics/
├── data/
│   └── students.csv       # Dataset (20 records)
├── main.py                # Runs the project and prints the analysis
├── functions.py           # Reusable calculation functions
├── requirements.txt       # pandas, numpy
├── README.md              # Project overview and how to run
└── Project_Documentation.md   # This document
```

### Step 1 – Data loading

`main.py` loads the CSV into a Pandas DataFrame and cleans the column names:

```
df = pd.read_csv("data/students.csv")
df.columns = df.columns.str.strip()
```

The data is then displayed so the columns and number of records can be checked.

### Step 2 – Total and average marks

Two functions in `functions.py` are used:

- `calculate_total_marks(dataframe)` – adds Python, Data Structures and Java marks.
- `calculate_average(total_marks)` – divides the total by 3 and rounds to 2 decimals.

The results are stored as new DataFrame columns: `Total_Marks` and `Average_Marks`.

### Step 3 – Grade and Pass/Fail status

The functions `calculate_grade()` and `calculate_status()` are applied to the average marks using `.apply()`.

**Grading rule**

| Average Marks | Grade |
|---|---|
| 90 and above | A+ |
| 80 to below 90 | A |
| 70 to below 80 | B |
| 60 to below 70 | C |
| 50 to below 60 | D |
| Below 50 | F |

**Pass/Fail rule:** Average marks ≥ 50 → **Pass**, otherwise **Fail**.

Note: For this project, the Pass/Fail decision is based on average marks. This is a simplified rule used for analysis and may differ from actual institutional examination rules.

### Step 4 – Class performance

`calculate_class_statistics()` uses Pandas (`max()`, `min()`, `mean()`) and boolean filtering (`Status == "Pass"`) to find the highest, lowest and overall average marks and the number of passed and failed students.

### Step 5 – Subject-wise analysis

`calculate_subject_averages()` returns a dictionary with the average of each subject. A `for` loop in `main.py` prints each subject's average, and `max()` finds the best-performing subject.

### Step 6 – Top performers

`get_top_students()` sorts the DataFrame by `Average_Marks` in descending order and returns the top 5 students using `.head(5)`.

### Step 7 – Attendance analysis

- `calculate_attendance_statistics()` returns average, highest and lowest attendance.
- `get_low_attendance_students()` filters students whose attendance is below 75%.

### Step 8 – NumPy statistical analysis

`calculate_numpy_statistics()` converts the `Average_Marks` column to a NumPy array using `.to_numpy()` and calculates `np.mean()`, `np.max()`, `np.min()` and `np.std()`.

### Step 9 – Final output

`main.py` prints every result under a clear heading (e.g. `===== CLASS PERFORMANCE ANALYSIS =====`) so the output can be understood without reading the code.

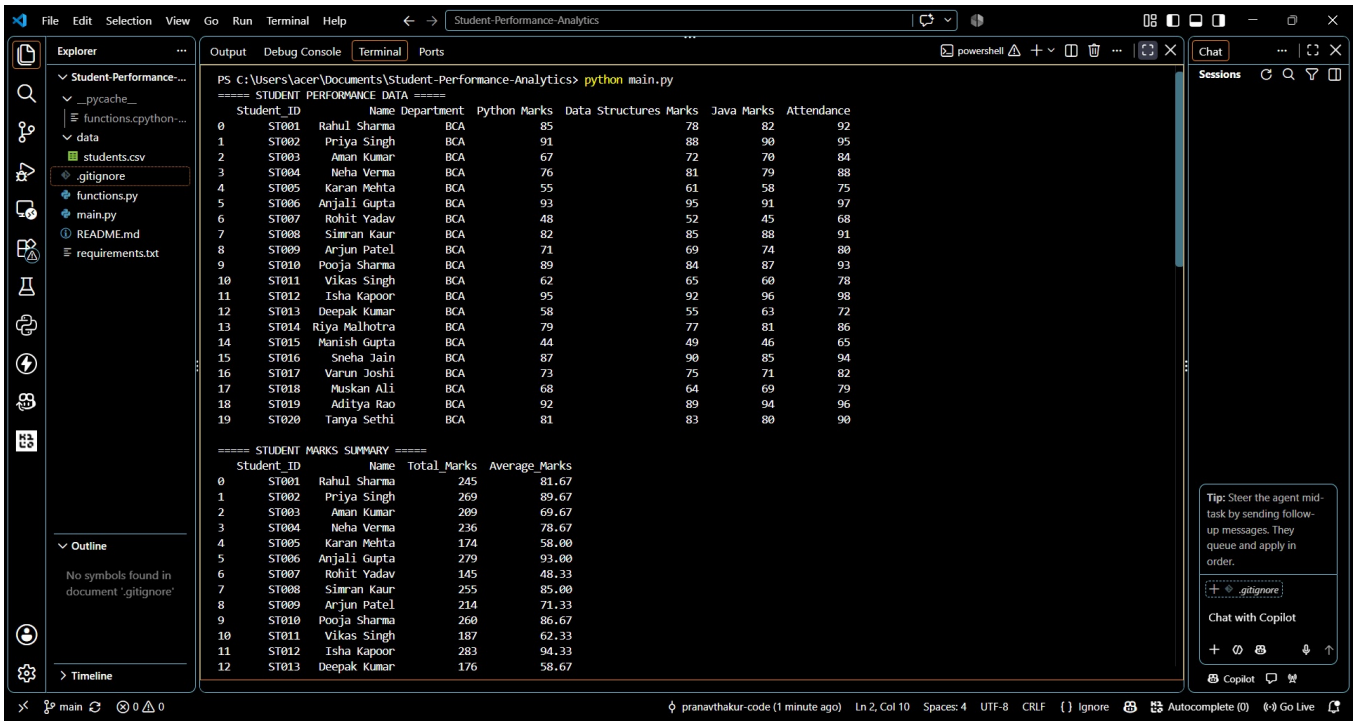
## 6. Key Features

| Function / Feature | File | Description |
|---|---|---|
| `calculate_total_marks()` | functions.py | Total marks of each student |
| `calculate_average()` | functions.py | Average marks of each student |
| `calculate_grade()` | functions.py | Assigns grade (A+ to F) |
| `calculate_status()` | functions.py | Assigns Pass / Fail |
| `calculate_class_statistics()` | functions.py | Highest, lowest, class average, pass/fail count |
| `calculate_subject_averages()` | functions.py | Subject-wise averages |
| `get_top_students()` | functions.py | Top 5 students by average marks |
| `calculate_attendance_statistics()` | functions.py | Average, highest, lowest attendance |
| `get_low_attendance_students()` | functions.py | Students with attendance below 75% |
| `calculate_numpy_statistics()` | functions.py | Mean, max, min, standard deviation using NumPy |
| Modular design | main.py + functions.py | Logic and display are kept separate |

## 7. Output / Screenshots

The project was run in VS Code using `python main.py` from the project root folder. The complete output of the program is shown in the screenshots below.

**Screenshot 1 – Running `python main.py`: student dataset loaded with Pandas and the start of the marks summary**



**Screenshot 2 – Total and average marks of every student, and the start of the grade and result table**




```
===== STUDENT MARKS SUMMARY =====
   Student_ID           Name  Total_Marks  Average_Marks
0       ST001   Rahul Sharma          245          81.67
1       ST002    Priya Singh          269          89.67
2       ST003     Aman Kumar          209          69.67
3       ST004     Neha Verma          236          78.67
4       ST005    Karan Mehta          174          58.00
5       ST006   Anjali Gupta          279          93.00
6       ST007    Rohit Yadav          145          48.33
7       ST008    Simran Kaur          255          85.00
8       ST009    Arjun Patel          214          71.33
9       ST010   Pooja Sharma          260          86.67
10      ST011    Vikas Singh          187          62.33
11      ST012    Isha Kapoor          283          94.33
12      ST013   Deepak Kumar          176          58.67
13      ST014  Riya Malhotra          237          79.00
14      ST015   Manish Gupta          139          46.33
15      ST016     Sneha Jain          262          87.33
16      ST017    Varun Joshi          219          73.00
17      ST018     Muskan Ali          201          67.00
18      ST019     Aditya Rao          275          91.67
19      ST020    Tanya Sethi          244          81.33

===== GRADE & RESULT =====
   Student_ID           Name  Average_Marks Grade Status
0       ST001   Rahul Sharma          81.67     A   Pass
1       ST002    Priya Singh          89.67     A   Pass
2       ST003     Aman Kumar          69.67     C   Pass
3       ST004     Neha Verma          78.67     B   Pass
4       ST005    Karan Mehta          58.00     D   Pass
5       ST006   Anjali Gupta          93.00    A+   Pass
6       ST007    Rohit Yadav          48.33     F   Fail
7       ST008    Simran Kaur          85.00     A   Pass
8       ST009    Arjun Patel          71.33     B   Pass
9       ST010   Pooja Sharma          86.67     A   Pass
10      ST011    Vikas Singh          62.33     C   Pass
11      ST012    Isha Kapoor          94.33    A+   Pass
12      ST013   Deepak Kumar          58.67     D   Pass
```

**Screenshot 3 – Grade & result, class performance analysis (highest, lowest, average, pass/fail) and subject-wise performance**




```
===== GRADE & RESULT =====
   Student_ID           Name  Average_Marks Grade Status
0       ST001   Rahul Sharma          81.67     A   Pass
1       ST002    Priya Singh          89.67     A   Pass
2       ST003     Aman Kumar          69.67     C   Pass
3       ST004     Neha Verma          78.67     B   Pass
4       ST005    Karan Mehta          58.00     D   Pass
5       ST006   Anjali Gupta          93.00    A+   Pass
6       ST007    Rohit Yadav          48.33     F   Fail
7       ST008    Simran Kaur          85.00     A   Pass
8       ST009    Arjun Patel          71.33     B   Pass
9       ST010   Pooja Sharma          86.67     A   Pass
10      ST011    Vikas Singh          62.33     C   Pass
11      ST012    Isha Kapoor          94.33    A+   Pass
12      ST013   Deepak Kumar          58.67     D   Pass
13      ST014  Riya Malhotra          79.00     B   Pass
14      ST015   Manish Gupta          46.33     F   Fail
15      ST016     Sneha Jain          87.33     A   Pass
16      ST017    Varun Joshi          73.00     B   Pass
17      ST018     Muskan Ali          67.00     C   Pass
18      ST019     Aditya Rao          91.67    A+   Pass
19      ST020    Tanya Sethi          81.33     A   Pass

===== CLASS PERFORMANCE ANALYSIS =====
Highest Average: 94.33
Lowest Average: 46.33
Overall Class Average: 75.15
Number of Passed Students: 18
Number of Failed Students: 2

===== SUBJECT-WISE PERFORMANCE =====
Python Average: 74.8
Data Structures Average: 75.2
Java Average: 75.45
Best Performing Subject: Java
Best Subject Average: 75.45

===== TOP 5 PERFORMING STUDENTS =====
```

**Screenshot 4 – Subject-wise result, Top 5 students, attendance analysis, low-attendance students and NumPy statistics**




## 8. Final Outcome

The project successfully reads a 20-record student dataset and produces a complete performance report. It achieved the following:

- Calculated total and average marks for all 20 students.
- Assigned grades and Pass/Fail status using clearly defined rules: **18 students passed and 2 failed**.
- Found the highest average (**94.33**, Isha Kapoor), lowest average (**46.33**, Manish Gupta) and overall class average (**75.15**).
- Found that **Java** is the best-performing subject (average 75.45), with all three subjects close to each other (74.8 – 75.45).
- Identified the top 5 students, with Isha Kapoor, Anjali Gupta and Aditya Rao all scoring A+.
- Found that 3 students have attendance below 75%, and observed that the 2 failing students are also among the low-attendance students.
- Used NumPy to calculate the standard deviation (14.23), which shows how widely the marks are spread.

The code is organised into reusable functions, runs without errors, and prints output that is clear without inspecting the code.

## 9. Challenges & Learning

| Challenge | How it was solved |
|---|---|
| Some column names in the CSV had extra spaces (e.g. `Python Marks `), which can cause `KeyError` when accessing columns. | Used `df.columns = df.columns.str.strip()` right after loading the data. |
| Deciding a clear Pass/Fail condition. | Defined a simple rule: average marks ≥ 50 is Pass. The rule is written in one function, so it is easy to change. |
| The program must find `data/students.csv`. | Used a relative path and run `python main.py` from the project root folder (also mentioned in the README). |
| Keeping `main.py` readable as features increased. | Moved all calculations into `functions.py` and kept only loading and printing in `main.py`. |

**What I learned**

- Reading CSV data and working with Pandas DataFrames.
- Creating new columns, filtering and sorting data.
- Writing reusable functions and organising code in multiple files.
- Using NumPy arrays for statistical calculations.
- Using conditional statements for grading and Pass/Fail logic.
- Preparing a project, README and documentation for GitHub.

**Possible future improvements:** charts using Matplotlib, department-wise comparison, more subjects and records, and exporting the report to a file.

## 10. GitHub Repository

https://github.com/pranavthakur-code/Student-Performance-Analytics